

UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES



REDES DE COMPUTADORAS

Trabajo práctico N°3

Docente: Santiago Martin Henn

Nombre de Grupo: Death Net

Integrantes:

- Alvarez Quiñones, Andrés
- Amarante, Flavio Eduardo
- Borch, Pehuen Emmanuel
- Carreras, Leonor Victoria
- Hernandez Monagas, Estefany

DESARROLLO:

1)

- a) La capa de enlace de datos (segunda capa del modelo OSI) se encarga de transferir información de manera confiable entre dos equipos conectados al mismo enlace o red local. Para lograrlo, organiza los bits en tramas, utiliza direcciones físicas MAC, controla el acceso al medio y detecta errores producidos durante la transmisión, resolviendo la comunicación nodo a nodo. En otras palabras, permite el envío de tramas entre dispositivos vecinos, como por ejemplo desde una computadora hacia un router.
- b) Como se mencionó en el punto “a”, una dirección MAC es un identificador físico asociado a la interfaz de red de un dispositivo. Pertenecce a la capa de enlace del modelo OSI y se utiliza para identificar y comunicar dispositivos dentro de una misma red local. Se representa en formato hexadecimal y generalmente contiene 48 bits. La diferencia principal radica en que la dirección MAC identifica a la interfaz física dentro de la red local, mientras que la dirección IP identifica lógicamente al dispositivo y la red a la que pertenece, permitiendo la comunicación entre redes diferentes.
- c) Una trama Ethernet es la unidad de información (PDU) utilizada en la capa de enlace de datos para transmitir información entre dispositivos dentro de una misma red local. Sus campos principales son:
 - **Preámbulo:** Secuencia de 7 bytes que permite sincronizar el reloj entre el y el receptor.
 - **SFD (Delimitador de inicio de trama):** Un byte específico que indica el fin del preámbulo y señala el comienzo exacto de la trama.
 - **Dirección MAC de destino:** Identifica la interfaz de red que debe recibir la trama.
 - **Dirección MAC de origen:** Identifica la interfaz de red que envió la trama.
 - **Tipo/Longitud:** Según el formato de la trama Ethernet, este campo puede indicar la longitud de los datos o identificar el protocolo de capa superior encapsulado. En Ethernet II, funciona como EtherType, indicando protocolos como IPv4 o IPv6.
 - **Datos o carga útil:** Contiene la información que se desea transportar, normalmente un paquete de la capa de red. Puede ocupar entre **46 y 1500 bytes**.
 - **Relleno (Padding):** Se añade automáticamente cuando la carga útil es menor a 46 bytes para alcanzar el tamaño mínimo requerido para la trama (64 bytes en total).

- **FCS (Secuencia de verificación de trama):** Contiene un código de redundancia cíclica (CRC) que permite al receptor detectar errores producidos durante la transmisión.

d) La información que permite determinar qué protocolo de capa superior está transportando una trama Ethernet es el campo Tipo o EtherType. Este es un valor de 2 bytes que indica cómo debe interpretarse la carga útil (por ejemplo, 0x0800 para IPv4, 0x86DD para IPv6 o 0x0806 para ARP).

2)

No.	Time	Source	Destination	Protocol	Length	Info
5747	60.347202...	74.125.250.254	192.168.0.193	UDP	1237	3478 → 49735 Len=1195
5748	60.348303...	74.125.250.254	192.168.0.193	UDP	1237	3478 → 49735 Len=1195
5749	60.348305...	74.125.250.254	192.168.0.193	UDP	253	3478 → 49735 Len=211
5750	60.348305...	74.125.250.254	192.168.0.193	UDP	257	3478 → 49735 Len=215
5751	60.348306...	74.125.250.254	192.168.0.193	UDP	1237	3478 → 49735 Len=1195
5752	60.351633...	74.125.250.254	192.168.0.193	UDP	1237	3478 → 49735 Len=1195
5753	60.386172...	74.125.250.254	192.168.0.193	UDP	281	3478 → 49735 Len=239
5754	60.393744...	192.168.0.193	34.214.215.20	TCP	55	57413 → 11111 [ACK] Seq=1 Ack=1 Win=65413 Len=1


```

Frame 5753: Packet, 281 bytes on wire (2248 bits), 281 bytes captured (2248 bits) on 0
Ethernet II, Src: SagemcomBroa_9e:c0:b9 (44:05:3f:9e:c0:b9), Dst: Intel_2a:94:dc (dc:21:5c:2a:94:dc)
  Destination: Intel_2a:94:dc (dc:21:5c:2a:94:dc)
    ....0. .... = LG bit: Globally unique address (factory default)
    ....0. .... = IG bit: Individual address (unicast)
  Source: SagemcomBroa_9e:c0:b9 (44:05:3f:9e:c0:b9)
    ....0. .... = LG bit: Globally unique address (factory default)
    ....0. .... = IG bit: Individual address (unicast)
  Type: IPv4 (0x0800)
    [Stream index: 0]
  Internet Protocol Version 4, Src: 74.125.250.254, Dst: 192.168.0.193
  User Datagram Protocol, Src Port: 3478, Dst Port: 49735
  Data (239 bytes)
    0000  dc 21 5c 2a 94 dc 44 05 3f 9e c0 b9 08 00 45 00  ..N*..D. ?....E
    0010  01 0b 99 5f 00 00 78 11 a1 9d 4a 7d fa fe c0 a8  ....x...J}....
    0020  00 c1 0d 96 c2 47 00 f7 87 3e 92 3f 28 85 2e 4b  ....G...>?(...K
    0030  66 51 00 00 1a 0c 65 35 09 22 00 00 0a be de f0  ....e5 "...*...
    0040  00 06 31 26 dc 10 97 9f ee 3b 2d ac 06 f3 0a 4e  ..18...;...N
    0050  ff ff ff ff 40 ff da 41 00 00 ca f1 60 4f c6 75  ...@..A...O..u
    0060  3d f7 8d c8 94 ce cd e9 5b 9d f5 18 b5 53 05 5f  ...[...S...
    0070  d2 08 38 e6 ef de 0f 92 e7 8e ff d4 eb 96 47 69  ..8...G...I
    0080  9b cd 32 9f d7 13 aa f5 a2 e0 dd fc f5 28 28 f0  ..2...((...
    0090  22 e0 1a 7f 63 20 a9 ca c2 63 81 2d f0 f6 a4 b1  "...c...c...((
    00a0  dd 04 9a c7 77 1c 39 f7 84 1d 77 50 83 85 d4 12  ...w9...wP...
    00b0  82 d8 ad cd f4 12 32 ca 02 23 1a a7 f3 df 08 46  ...2...#....F
    00c0  0c 4e f8 44 5e c2 17 55 ee 58 f1 b6 e0 5c 6e 0d  ..N.D^..U..X...N
    00d0  dc f5 46 fd d4 98 06 9c 06 00 a8 76 4f c5 fd 8b  ..F...v0...
    00e0  6c ca a8 3f 9d 76 64 dc 56 37 8c 2e 2d 72 cc ae  1..?..vd..V7...n
    00f0  f6 31 8a 89 e1 46 03 f3 ad 17 17 b2 0c 2e 1f 46  ..1...F...F...
    0100  5f fe d1 37 ff e4 3a 93 0a b0 9f eb d1 bc 13 8e  ...7:.....
    0110  85 f7 a9 49 40 bb 4e 85 24  ....I@N.$
  
```

a) Dentro de la sección de Ethernet II se pueden encontrar las direcciones MAC de origen y de destino de la trama. En el caso de la trama seleccionada en la imagen anterior se tiene que las direcciones son las siguientes:

- Destino: dc:21:5c:2a:94:dc
- Origen: 44:05:3f:9e:c0:b9

▼ Ethernet II, Src: SagemcomBroa_9e:c0:b9 (44:05:3f:9e:c0:b9), Dst: Intel_2a:94:dc (dc:21:5c:2a:94:dc)	
▼ Destination: Intel_2a:94:dc (dc:21:5c:2a:94:dc)	
....0. = LG bit: Globally unique address (factory default)	
....0. = IG bit: Individual address (unicast)	
▼ Source: SagemcomBroa_9e:c0:b9 (44:05:3f:9e:c0:b9)	
....0. = LG bit: Globally unique address (factory default)	
....0. = IG bit: Individual address (unicast)	
Type: IPv4 (0x0800)	
[Stream index: 0]	

Para determinar a qué dispositivos corresponden estas direcciones, se verificó la dirección MAC de la interfaz inalámbrica de la notebook desde la cual se realizó la captura. La dirección MAC de la notebook coincide con la dirección de destino.

Por ende, la dirección MAC de destino de la trama seleccionada corresponde a la notebook, mientras que la dirección MAC de origen corresponde probablemente al router Wi-Fi.

- b) Dentro de la sección a continuación (Internet Protocol Version 4), se pueden obtener las direcciones IP tanto de origen como de destino del paquete.
- Destino: 192.168.0.193
 - Origen: 74.125.250.254

```
▼ Internet Protocol Version 4, Src: 74.125.250.254, Dst: 192.168.0.193
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  ▶ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 267
    Identification: 0x995f (39263)
  ▶ 000. .... = Flags: 0x0
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 120
    Protocol: UDP (17)
    Header Checksum: 0xa19d [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 74.125.250.254
    Destination Address: 192.168.0.193
    [Stream index: 0]
```

- c) Como se puede confirmar al comparar entre los apartados a y b, la dirección IP y MAC no son iguales y, fundamentalmente, no representan lo mismo. Cumplen diferentes funciones y pertenecen a distintas capas de la arquitectura de red.
- En la trama analizada, las direcciones MAC de origen y destino corresponden al router Sagemcom y a la interfaz de red Intel de la notebook respectivamente. Por otro lado, las direcciones IP de origen y destino corresponden a una dirección IP pública perteneciente al dispositivo remoto que envía los datos, mientras que la segunda corresponde a la dirección IP privada de la notebook dentro de la red local.
- d) En la sección Ethernet II, dentro del campo *Type* se presenta el valor 0x0800 junto con la aclaración IPv4 (Internet Protocol Version 4). Indicando que el protocolo encapsulado dentro de la trama Ethernet analizada es IPv4. También se puede corroborar en la sección que se vio en el punto b puesto que el nombre de dicha sección especifica el versionado del protocolo de Internet.

3)

- a) TCP proporciona una comunicación confiable y orientada a conexión entre aplicaciones, algo que Ethernet e IP no garantizan por sí mismos. Se encarga de asegurar que todos los datos lleguen correctamente y en el mismo orden en que fueron enviados.
- Si algún segmento se pierde o llega dañado, TCP gestiona su retransmisión. Además, controla la velocidad de transmisión (control de flujo) para no saturar al receptor y utiliza números de puerto para entregar los datos a la aplicación correspondiente.
- b) La cabecera TCP contiene información necesaria para controlar la comunicación. Sus campos más importantes son:

- **Puerto de origen y destino:** Identifican las aplicaciones o procesos específicos en el emisor y en el receptor que están enviando y recibiendo los datos.
- **Número de secuencia (Sequence Number):** Indica la posición del primer byte de datos de este segmento en el flujo total de datos, permitiendo al receptor reordenarlos correctamente.
- **Número de confirmación (ACK Number):** Informa cuál es el próximo byte que el receptor espera recibir, confirmando así la recepción correcta de los bytes anteriores.
- **Longitud de la cabecera (Data Offset):** Especifica el tamaño de la cabecera TCP para saber dónde empiezan realmente los datos de la aplicación.
- **Banderas de control (Flags):** indican el estado o la función del segmento. Las principales son “SYN” para iniciar una conexión, “ACK” para confirmar información, “FIN” para finalizar y “RST” para reiniciarla.
- **Tamaño de ventana (Window Size):** indica cuántos datos puede recibir el equipo sin saturarse.
- **Suma de comprobación (Checksum):** permite detectar errores en la cabecera y en los datos.

Luego de la cabecera TCP, se encuentra la carga útil, que contiene la información enviada por la aplicación.

- c) El three-way handshake es el proceso utilizado para iniciar una conexión TCP y consta de tres pasos:
- 1) **SYN:** El cliente envía un segmento con la bandera **SYN** para solicitar el inicio de la conexión.
 - 2) **SYN-ACK:** El servidor responde con **SYN-ACK**, aceptando la solicitud y confirmando la recepción del segmento.
 - 3) **ACK:** El cliente envía un **ACK** para confirmar la respuesta del servidor.

Tras estos tres pasos, la conexión queda establecida y pueden comenzar el intercambio de datos.

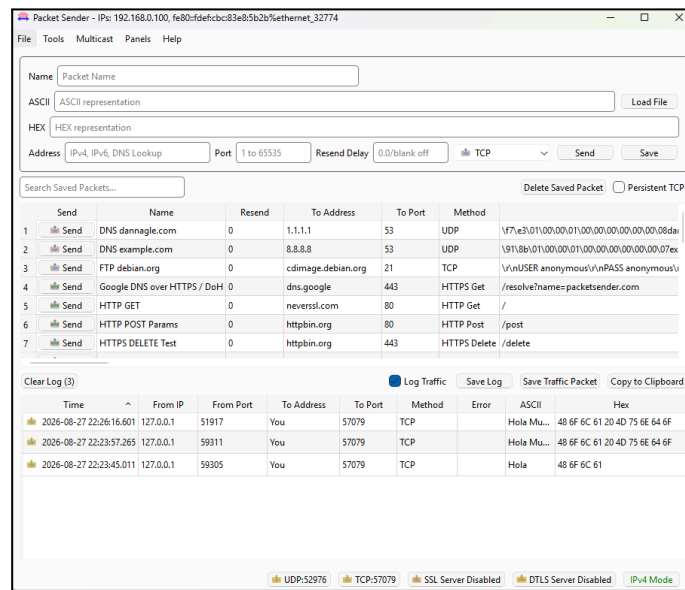
Por otro lado, el Four-way handshake se utiliza para finalizar la conexión de forma ordenada:

- 1) El cliente envía un segmento **FIN** para indicar que no enviará más datos.
- 2) El servidor responde con un **ACK** para confirmar la recepción del cierre.

- 3) Cuando el servidor concluye sus transmisiones pendientes, envía su propio FIN.
- 4) El cliente confirma la recepción del servidor con un último ACK.

Se necesitan cuatro pasos porque TCP es una comunicación full-duplex (bidireccional), por lo que cada sentido del flujo de datos se cierra de forma independiente.

d)



tcp.dstport == 57079					
No.	Time	Source	Destination	Protocol	Length Info
70	10.991406100	127.0.0.1	127.0.0.1	TCP	56 51917 → 57079 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
71	10.991475700	127.0.0.1	127.0.0.1	TCP	56 57079 → 51917 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
72	10.991507700	127.0.0.1	127.0.0.1	TCP	44 51917 → 57079 [ACK] Seq=1 Ack=1 Win=65280 Len=0
73	11.009623500	127.0.0.1	127.0.0.1	TCP	54 51917 → 57079 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=10
74	11.009655400	127.0.0.1	127.0.0.1	TCP	44 57079 → 51917 [ACK] Seq=1 Ack=11 Win=65280 Len=0
75	11.012199600	127.0.0.1	127.0.0.1	TCP	44 57079 → 51917 [FIN, ACK] Seq=1 Ack=11 Win=65280 Len=0
76	11.012236400	127.0.0.1	127.0.0.1	TCP	44 51917 → 57079 [ACK] Seq=11 Ack=2 Win=65280 Len=0
77	11.012281700	127.0.0.1	127.0.0.1	TCP	44 51917 → 57079 [FIN, ACK] Seq=11 Ack=2 Win=65280 Len=0
78	11.012312600	127.0.0.1	127.0.0.1	TCP	44 57079 → 51917 [ACK] Seq=2 Ack=12 Win=65280 Len=0

> Frame 73: Packet, 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface	0000 02 00 00 00 45 00 00 32 4a 5d 40 00 80 06 b2 66	E..2 J]@....f
> Null/Loopback	0010 7f 00 00 01 7f 00 00 01 ca cd de f7 3f 75 4c e9	?uL.....
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1	0020 ff f3 43 17 50 18 00 ff 88 96 00 00 48 6f 6c 61	..C.P.....Holo
> Transmission Control Protocol, Src Port: 51917, Dst Port: 57079, Seq: 1, Ack: 1, Len: 10	0030 20 4d 75 6e 64 6f	Mundo
> Data (10 bytes)		

Analizando el paquete de datos en Wireshark, se pudo observar que la comunicación se realizó dentro de la misma computadora, por eso la dirección IP de origen y destino fueron el 127.0.0.1 (localhost). El segmento TCP salió desde el puerto 51917 y llegó al puerto 57079, donde se encontraba funcionando el servidor. Además, presentaba las banderas PSH y ACK, que indican que se

estaban enviando datos y confirmando la información recibida. Por último, en la sección de datos se encontró una carga útil de 10 bytes. Wireshark permitió verla en formato hexadecimal y también como ASCII, donde se pudo reconocer el mensaje enviado.

e)

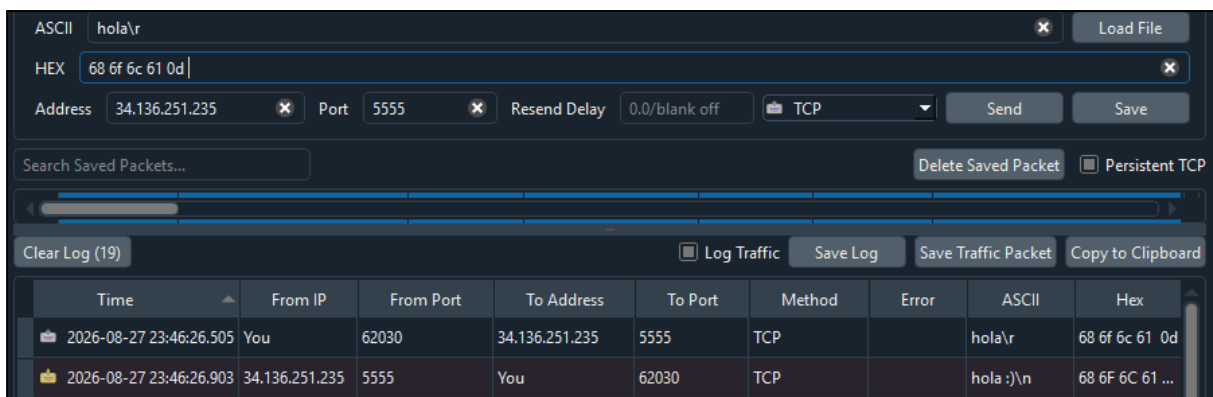
75	11.012199600	127.0.0.1	127.0.0.1	TCP	44 57079 → 51917 [FIN, ACK] Seq=1 Ack=11 Win=65280 Len=0
76	11.012236400	127.0.0.1	127.0.0.1	TCP	44 51917 → 57079 [ACK] Seq=11 Ack=2 Win=65280 Len=0
77	11.012281700	127.0.0.1	127.0.0.1	TCP	44 51917 → 57079 [FIN, ACK] Seq=11 Ack=2 Win=65280 Len=0
78	11.012312600	127.0.0.1	127.0.0.1	TCP	44 57079 → 51917 [ACK] Seq=2 Ack=12 Win=65280 Len=0

Al finalizar el envío del mensaje, la conexión TCP se cerró automáticamente y el proceso pudo observarse en Wireshark. Primero, el servidor envió un segmento con las banderas **FIN** y **ACK**, indicando que quería finalizar su parte de la comunicación. El cliente confirmó su recepción mediante un **ACK**. Luego, el cliente envió su propio segmento **FIN** y **ACK**, y el servidor respondió con un último **ACK**, dejando la conexión completamente cerrada.

Este proceso se denomina **Four-way handshake** y utiliza cuatro pasos porque cada sentido de la comunicación TCP debe cerrarse por separado. En la captura se puede observar este intercambio en los paquetes 75, 76, 77 y 78.

f) La facilidad con la que los paquetes viajan a través de la red permite trasladar información de forma rápida y eficiente. Sin embargo, también puede generar problemas de seguridad cuando los datos se transmiten sin cifrado, ya que una persona que consiga acceder al tráfico podría observar mensajes, direcciones, puertos y otra información sensible. Por esta razón, es importante utilizar protocolos seguros, como HTTPS, que cifran el contenido y protegen la información transportada en los paquetes.

- 4) Se utilizó Packet Sender como cliente TCP para establecer una conexión con un servidor ubicado en la nube. La comunicación se realizó utilizando la dirección IP 34.136.251.235 y el puerto 5555. La captura del tráfico generado durante la conexión se realizó mediante Wireshark.



Al establecer conexión con el servidor, se enviaron los comandos indicados en la consigna. El intercambio de los comandos y las respuestas del servidor son las documentadas a continuación:

```

TCP://34.136.251.235:5555
Connected and idle.
DNS dannagle.com Load Search Packets... Raw ASCII
ASCII. Press Enter to Send. Send Append \r Send File

hola
hola :)
ping
pong
tic
toc
status
leyendo tu historial de búsquedas (que horror, buscá ayuda profesional)
status
esperando comando
status
comiendo flynn-paff
status
esperando comando
Death Net
seq: 6, payload: w.

00:05:03 Clear Log To Clipboard Cerrar

```

El último comando corresponde al grupo y permite obtener el número de secuencia y la carga útil asignados.

Para analizar el intercambio de paquetes desde Wireshark se utilizó un filtro (*tcp.port == 5555 && ip.addr == 34.136.251.235*) para obtener los paquetes correspondientes a la comunicación entre la computadora y el servidor.

La notebook utilizada como cliente tiene dirección IP 192.168.0.193, mientras que el servidor posee la dirección IP pública 34.136.251.235. La comunicación utiliza el puerto 5555 del servidor.

3225	318.668303100	192.168.0.193	34.136.251.235	TCP	66	63262 → 5555 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3226	318.888396200	34.136.251.235	192.168.0.193	TCP	66	5555 → 63262 [SYN, ACK] Seq=0 Ack=1 Win=65320 Len=0 MSS=1420 SACK_PERM WS=128
3227	318.888880400	192.168.0.193	34.136.251.235	TCP	54	63262 → 5555 [ACK] Seq=1 Ack=1 Win=65280 Len=0
3228	318.889826000	192.168.0.193	34.136.251.235	TCP	58	63262 → 5555 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=4
3229	319.094283300	34.136.251.235	192.168.0.193	TCP	60	5555 → 63262 [ACK] Seq=1 Ack=5 Win=65408 Len=0
3230	319.094285300	34.136.251.235	192.168.0.193	TCP	60	5555 → 63262 [PSH, ACK] Seq=1 Ack=5 Win=65408 Len=4
3231	319.145635600	192.168.0.193	34.136.251.235	TCP	54	63262 → 5555 [ACK] Seq=5 Ack=5 Win=65280 Len=0

Al establecer la conexión con el servidor se puede observar en Wireshark el **Three-way handshake** de TCP. La notebook envía un segmento con la bandera **SYN** al servidor, este responde con un segmento **SYN-ACK**, y finalmente el cliente envía un **ACK**, completando la conexión TCP.

De forma adicional, se recopilaron los payloads de cada grupo para poder concatenarlos respetando el orden de SEQ para obtener la siguiente cadena:

[“https://www.youtube.com/watch?v=dQw4w9WgXcQ”](https://www.youtube.com/watch?v=dQw4w9WgXcQ)

```
seq: 1, payload: ht
seq: 2, payload: tp
seq: 3, payload: s:
seq: 4, payload: //
seq: 5, payload: ww
seq: 6, payload: w.
seq: 7, payload: yo
seq: 8, payload: u
seq: 9, payload: t
seq: 10, payload: ub
seq: 11, payload: e
seq: 12, payload: .c
seq: 13, payload: om
seq: 14, payload: /w
seq: 15, payload: a
seq: 16, payload: t
seq: 17, payload: ch
seq: 18, payload: ?
seq: 19, payload: v=
seq: 20, payload: d
seq: 21, payload: Qw
seq: 22, payload: 4w
seq: 23, payload: 9W
seq: 24, payload: gX
seq: 25, payload: cQ
```

REFERENCIAS

1. Stallings, W. (2004). *Comunicaciones y redes de computadores* (7.^a ed.). Pearson Educación.